

Spring Boot Image handling : project objective

Objective : Image upload n download --from server side folder

Use multipart request

An HTTP multipart request is an HTTP request that HTTP clients construct to send files and data over to an HTTP Server. It is commonly used by browsers and HTTP clients to upload files to the server.

`org.springframework.web.multipart.MultipartFile => i/f`

A representation of an uploaded file n it's metadata , received in a multipart request.

The file contents are either stored in memory or temporarily on disk. In either case, the programmer is responsible for copying file contents to a persistent store. The temporary storage will be cleared at the end of request processing.

Steps

0. Add a property in Entity (n DTO) , to store the image path

eg : `private String imagePath;`

1. To upload a file to a server side folder add the property (any property name)

`file.upload.location=images/` --add in `application.properties`

(This will be a folder relative to current project)

#limiting max file size for upload

`spring.servlet.multipart.max-file-size=10MB`

`spring.servlet.multipart.max-request-size=11MB`

2. For File upload

Create "images" folder , if not present

Create `ImageHandlingService` : as singleton n eager service bean

n create "images" folder , if not present

API of `java.io.File`

2.1 `File(String path)`

2.2 `exists`

2.3 `mkdir`

3. Create a rest controller

Other User details are already stored in DB. image path has to be stored in the table n image should be uploaded in the "images" folder.

i/p : User id : request param / path variable

`MultipartFile` : request param

resp : `ApiResponse` wrapping a message

eg : Method : POST

`http://localhost:8080/api/Userloyees/images`

Add in `@PostMapping` : `consumes = "multipart/form-data"`, along with value , for swagger .

4. Image Handling Service

`FileUtils` class from Apache commons-io

<!-- <https://mvnrepository.com/artifact/commons-io/commons-io> -->

```
<dependency>
<groupId>commons-io</groupId>
<artifactId>commons-io</artifactId>
<version>2.11.0</version>
</dependency>
```

Methods :

1. public static void writeByteArrayToFile(File file,byte[] data)
throws IOException

Writes a byte array to a file creating the file if it does not exist.

5. Save image path in User entity (to trigger update query) --setter

6. Return ApiResponse DTO to the caller.(service --> controller --> JSON representation to clnt)

5. Test it with postman or swagger or browser.

2. For Serving images (download) from server side folder.

i/p : User id

Steps

3.1 In Controller

Method = GET

Add "produces" element in the annotation(@GetMapping): to include image MediaType : JPEG_VALUE , GIF_VALUE, PNG_VALUE

eg : @GetMapping(value = "{/userId}/images", produces = { MediaType.IMAGE_GIF_VALUE, MediaType.IMAGE_JPEG_VALUE,MediaType.IMAGE_PNG_VALUE })

3.2 In service layer

1. get User details from User id

2. Get complete path of the image

User's getter

Check for non null. If null raise suitable exception (eg : ResourceNotFoundException)

3. FileUtils class API

public static byte[] readFileToByteArray(File file) throws IOException

Reads the contents of a file into a byte array. The file is always closed.

4. Return contents of image file(byte[]) to the controller

From controller , simply add it in ResponseEntity n send it to the clnt.

4. For react frontend , use : in : src as

Method : GET

http://localhost:8080/Userloyees/{UserId}/image